					2.4 and 3.2.1.1.7 - automatic deduction	SET-UP: Your user agreement in section 2.4 says three things: contributions cannot be reversed, the system can deduct overdue amounts from a member's financial accounts, and members who do not comply can be reported to the police. Section 3.2.1.1.7 then makes that automatic deduction a functional requirement - when a member misses a due date, the system deducts the money from their accounts. QUESTION: Which mobile money or bank interface lets your application take money out of someone's account without that person approving that	Concedes that no ordinary mobile money or bank API permits a third party to debit an account without per-transaction authorisation from the account holder. Identifies the only real routes: a standing mandate or direct debit arranged with the provider, or a pre-funded wallet the member tops up in advance - both of which need commercial agreements and probably a regulated status, which links straight back to their own Reserve Bank of Malawi constraint in 3.4. On the second half, recognises that withholding collateral and reporting someone to the police are things the group's members do, not things software does, and that they belong in a group constitution or terms of service rather than in section 3.2. Strong answers say plainly that the requiremen

					3.2.1.1.1 - login photographs	SET-UP: Section 3.2.1.1.1 says that at every login the system shall require camera access, capture a picture of the user with the front camera in the background, and save that picture in the backend. Section 3.4 says the system shall comply with Malawi's data protection laws. QUESTION: What is that photograph for, who is allowed to see it, and how long is it kept - and can you take someone's picture in the background at every login and still meet the data protection constraint you wrote yourself?	Recognises that the document never states a purpose for the photographs, which is the first thing data protection law asks. Understands that 'in the background' means without the user's awareness at that moment, and that this is the part that fails - the fix is explicit notice and consent at registration, a stated purpose (fraud evidence or identity assurance), a retention period, restricted access, encryption at rest and a deletion rule. Notes that none of these appears in 3.5.1 Security or anywhere else, and that no section 3.6 exists to hold them. Strong answers propose replacing a photograph at every login with an explicit liveness check at high-risk events only, and connect this to the National ID details the system also collects without any protection

					3.1 - missing interface subsections	SET-UP: The structure requires four interface subsections: user, hardware, software and communication. You wrote user interfaces, then payment, financial service and SMS interfaces - and those last three are all software interfaces. Meanwhile your system depends on a front camera, a facial depth sensor and a fingerprint scanner, and section 3.4 says the system works over an internet or USSD connection. QUESTION: Where should the camera, the depth sensor and the fingerprint scanner have been specified, and what belongs in a communication interfaces section for a system	Places the camera, depth sensor and fingerprint reader under Hardware Interfaces and specifies them properly: minimum camera capability, the biometric API used on each platform, and - importantly - what happens on a device with no depth sensor, since 2.3 assumes every super administrator has one. For communication interfaces: HTTPS with a named TLS version, the data format, the callback or webhook endpoint that receives payment confirmation, timeout and retry policy, and the SMS gateway protocol. Strong answers notice that the document names many providers but not one protocol, and that the USSD channel in 3.4 appears nowhere else - so either it is specified or the claim is removed.

					1.2 and 2.2 vs 3.1.3, 3.2.3.6, 3.2.3.7 - the investment activity	SET-UP: Your scope in 1.2 and your product functions in 2.2 say nothing about investing money. But section 3.1.3 specifies interfaces to Mpamba Fesa, Airtel Phuka and fixed bank accounts; 3.2.3.6 has the backend transferring member funds into them; and 3.2.3.7 records the interest earned as system profit. QUESTION: Is placing members' contributions into interest-earning accounts part of this project - and if it is, why does it appear nowhere in your scope, your product functions or your user agreement, and whose money is the interest?	Recognises this as a scope omission rather than a drafting slip, and that it is the most consequential one in the document: a whole capability with regulatory weight appears only in the detailed requirements. Answers the ownership question directly - the document says 'system profits' but the money placed is the members' contributions, and nothing in the 2.4 user agreement tells members their money will be invested or who keeps the return. Adds it to 1.2 and 2.2 and to the user agreement, and links it to the Reserve Bank of Malawi constraint in 3.4, since holding and investing other people's funds is regulated activity. Strong answers raise the liquidity risk: money tied up in a fixed deposit when a payout falls

					3.2.3.6 - fund placement	SET-UP: Requirement 3.2.3.6 says the backend shall intelligently decide which funds can be transferred into Fesa, Phuka and fixed accounts, and on what terms, in a way that will not affect any payout schedule. QUESTION: Rewrite that as something a developer could build and a tester could check - and tell me what happens if a fixed deposit has not matured on the day a payout is due.	Replaces 'intelligently' with explicit rules, because the word is the problem: a minimum liquid reserve covering at least the next scheduled payout or two, a maximum placement term that can never extend past the next payout date, a named role approving any placement, and a hard constraint that no placement may reduce the reserve below the threshold. On the maturity question, recognises that the current wording only asserts the schedule will not be affected without any mechanism to guarantee it, and adds an early-withdrawal or contingency path plus a rule that a payout is never delayed by a placement decision. Strong answers identify this as the highest-risk requirement in the document, because getting it wrong

					2.4 and 3.2.1.1.7 - voting rules	SET-UP: Section 2.4 says group actions requiring a vote shall only be taken if all voting members vote yes. Section 3.2.1.1.7 says the system shall allow group members to vote out a noncompliant member. Joining a group by request also requires acceptance from all current members. QUESTION: A member has stopped contributing and the group wants to remove them. Under your own rules, can that vote ever succeed - and what would you change?	Sees the deadlock straight away: the member facing removal is a voting member and will vote no, so a unanimity rule makes removal impossible, and the requirement to vote them out can never be satisfied. Sees the second case too - with up to twelve members, one inactive or unreachable member blocks every new joiner indefinitely. Proposes concrete fixes: exclude the subject of a vote from the count, set a majority threshold such as two-thirds of eligible members, define a quorum, and set a voting deadline with a stated default when members do not respond. Strong answers note that nothing in the document says what happens when members simply never vote, which is the commonest real outcome.

					2.5, 1.5 and document control	SET-UP: Section 2.5 lists as an assumption that the system 'shall do all its intended operations without making mistakes', while section 3.5.8 is an entire subsection on handling errors. Section 1.5 Overview re-describes the system instead of describing how the document is organised. And this document has no contents page, no version number, no date, no revision history and no approval block, while three of its headings are misspelled: PERFOMANCE, COSTRAINTS and SCALABLITY. QUESTION: Why is 'the system never makes mistakes' not an	Distinguishes an assumption - something outside the team's control that is taken as true, and becomes a risk if it turns out false - from a statement of hope about the team's own product, and sees that it contradicts not only 3.5.8 but also 3.5.2's requirements to prevent duplicate records and handle failed transactions. States that 1.5 Overview should be a roadmap of the document's own sections, and that as written it duplicates 1.2 and 2.1. On process: one named editor doing a final read-through, plus front matter with version, date, authors, reviewer and a revision history. Strong answers note the irony of a system built around traceable financial records arriving with no version history of its own.